27/04/2026

Rapport de Projet Logiciel
Méthodologie Agile Scrum

Start-up: SoukOnline

Leila ben Mehrez-Hamza amara-Ol El Khir Ghofrane Mahmoud-Lina Zghal-Abdelkader Jelassi
FACULTÉ DES SCIENCES ECONOMIQUES ET DE GESTION DE TUNIS

Table des matières

Souk Online : 2
Problème résolu et Cible Problème résolu 2
Acteurs du système : 2
Fonctionnalités principales : 2
Tableau complet des User Stories 3
Récapitulatif par priorité MoSCoW 3
Sprint 1 Goal : 4
Selections des User Stories (Must-Have): 4
Décomposition Technique : 4
Tableau Kanban: 4
Definition of Done (DoD): 4
Diagramme de cas d'utilisation (DCU) : 6
Description des cas d'utilisation : 6
Diagramme de classes (DC) : 7
Diagramme de séquence (DS) — Processus de commande 8
Description du flux : 8
Choix de conception : 9
Captures d'écran: 10
Lien Figma (prototype cliquable) : 15
Explication des parcours utilisateur 15
Méthodologie utilisée 16
- Leila (Product Owner + Dev) 21
- Hamza (Scrum Master + Dev) 22
- Ghofrane (Développeuse) 22
- Lina (Développeuse) 22
- Abdelkader (Développeur) 22

1. Fiche Projet

Souk Online :

Souk Online est une plateforme tout-en-un conçue pour centraliser l'intégralité du workflow des e-commerçants, de la gestion des commandes à la logistique de livraison. En regroupant les outils de communication, le suivi financier et l'automatisation des expéditions sur une interface unique, la solution permet de réduire la charge mentale des vendeurs et d'optimiser leur rentabilité.

Problème résolu et Cible Problème résolu :

Actuellement, les e-commerçants souffrent d'une fragmentation opérationnelle : ils doivent jongler entre Shopify pour les ventes, WhatsApp pour l'équipe, et divers portails de transporteurs pour les livraisons. Cette dispersion entraîne des erreurs de données, des coûts d'abonnement cumulés élevés et une perte de temps significative.
Cible : Les petits et moyens e-commerçants en Tunisie, les entrepreneurs en dropshipping et les boutiques locales cherchant à digitaliser leur logistique sans complexité technique.

Acteurs du système :

• E-commerçant (Admin) : Gère les produits, suit les profits et supervise l'équipe.

• Gestionnaire de commandes : Membre de l'équipe chargé du traitement des colis et de la communication client.

• Partenaire de Livraison (API) : Système tiers qui reçoit les demandes de ramassage et met à jour les statuts de livraison.

• Client Final : Interagit avec la boutique pour passer commande et reçoit les notifications de suivi.

Fonctionnalités principales :

Fonctionnalité	Responsable
Système d'authentification sécurisé et gestion des rôles	Leila
Module de paiement et calcul automatique du profit net	Hamza
Catalogue de produits dynamique avec gestion intelligente des stocks	Ghofrane
Gestionnaire centralisé du panier et cycle de vie des commandes	Lina
Tableau de bord administrateur avec statistiques de vente en temps réel	Abdelkader

2. Product Backlog

Tableau complet des User Stories

ID	Summary (Titre)	Description (User Story)	Priority (MoSCoW)	Story Points	Assignee
US-01	Inscription & Connexion	En tant qu'e-commerçant, je veux m'authentifier...	Must	3	Leila
US-02	Gestion d'équipe	En tant qu'admin, je veux inviter des collaborateurs...	Should	5	Leila
US-03	Simulation Paiement	En tant que client, je veux payer via une simulation...	Must	5	Hamza
US-04	Calcul Rentabilité	En tant qu'e-commerçant, je veux voir mon profit net...	Must	8	Hamza
US-05	Fiches Produits	En tant qu'e-commerçant, je veux gérer mes produits...	Must	3	Ghofrane
US-06	Alertes Stock	En tant qu'e-commerçant, je veux être alerté si stock bas...	Should	3	Ghofrane
US-07	Panier d'achat	En tant que client, je veux gérer mon panier...	Must	5	Lina
US-08	Suivi Livraison	En tant qu'e-commerçant, je veux mettre à jour le statut colis...	Must	5	Lina
US-09	Dashboard Ventes	En tant qu'e-commerçant, je veux voir mes graphiques de revenus...	Must	8	Abdelkader
US-10	Journal d'activités	En tant qu'admin, je veux voir l'historique des actions d'équipe...	Could	5	Abdelkader

Récapitulatif par priorité MoSCoW

Priorité	NB US	Points	User Stories
Must	7	29	US01,US03,US04,US05,US07,US08US09
Should	2	8	US02 , US06
Could	1	5	US10
Total	10	42	

3. Sprint Backlog

Sprint 1 Goal :

Mettre en place l'architecture technique et valider le cycle de vente complet (du catalogue au paiement).

Selections des User Stories (Must-Have):

✓ US-01 (Auth), US-03 (Paiement), US-04 (Profit), US-05 (Produits), US-07 (Panier).

Décomposition Technique :

Membre	Tache technique principale
Leila	Structure de la classe Utilisateur et session.
Hamza	Logique de calcul dans Finance et simulation CB.
Ghofrane	Classe Produit et persistance du catalogue.
Lina	Gestion de la liste d'objets dans Panier.
Abdelkader	Initialisation du Main et affichage console.

Tableau Kanban:

A' faire	En cours	En revue	Terminé
US-02, US-10	US-06, US-08	US-09	US-01, US-03 US-05, US-07

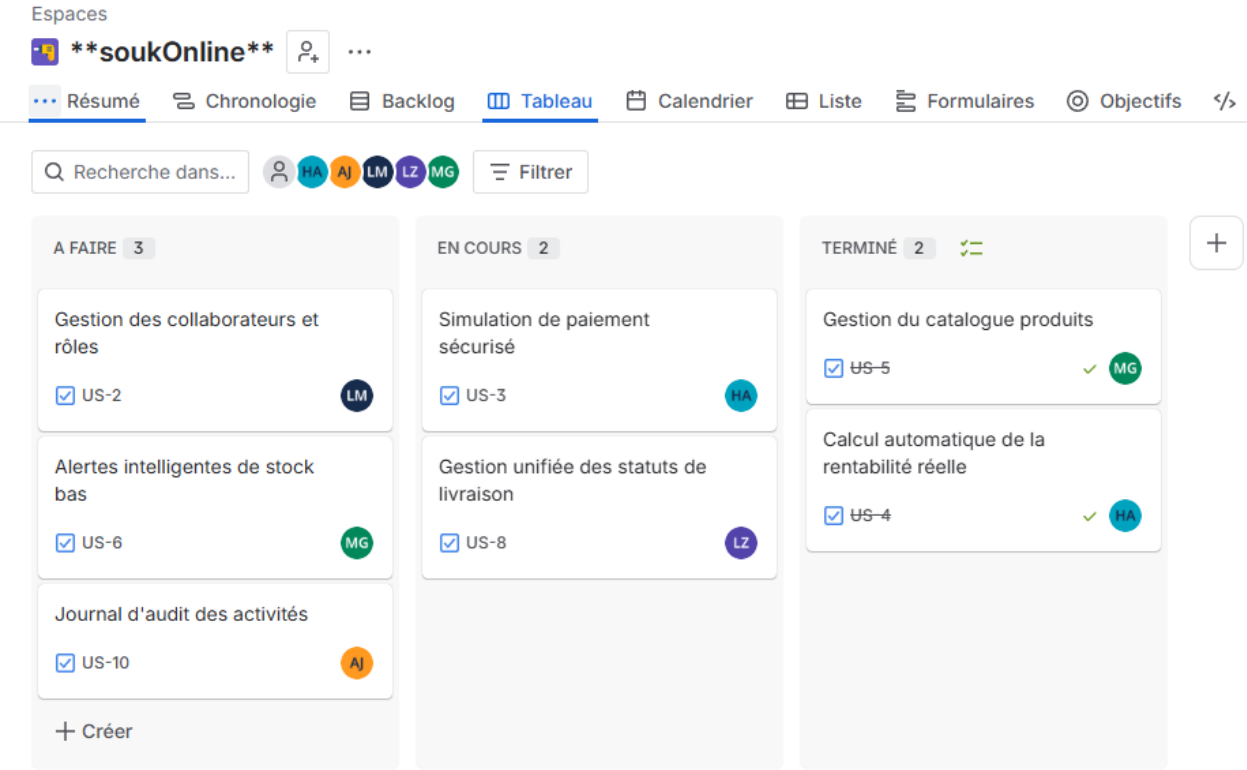
Definition of Done (DoD):

✓ Le code compile sans erreur.

✓ Le Main.java exécute le scénario.

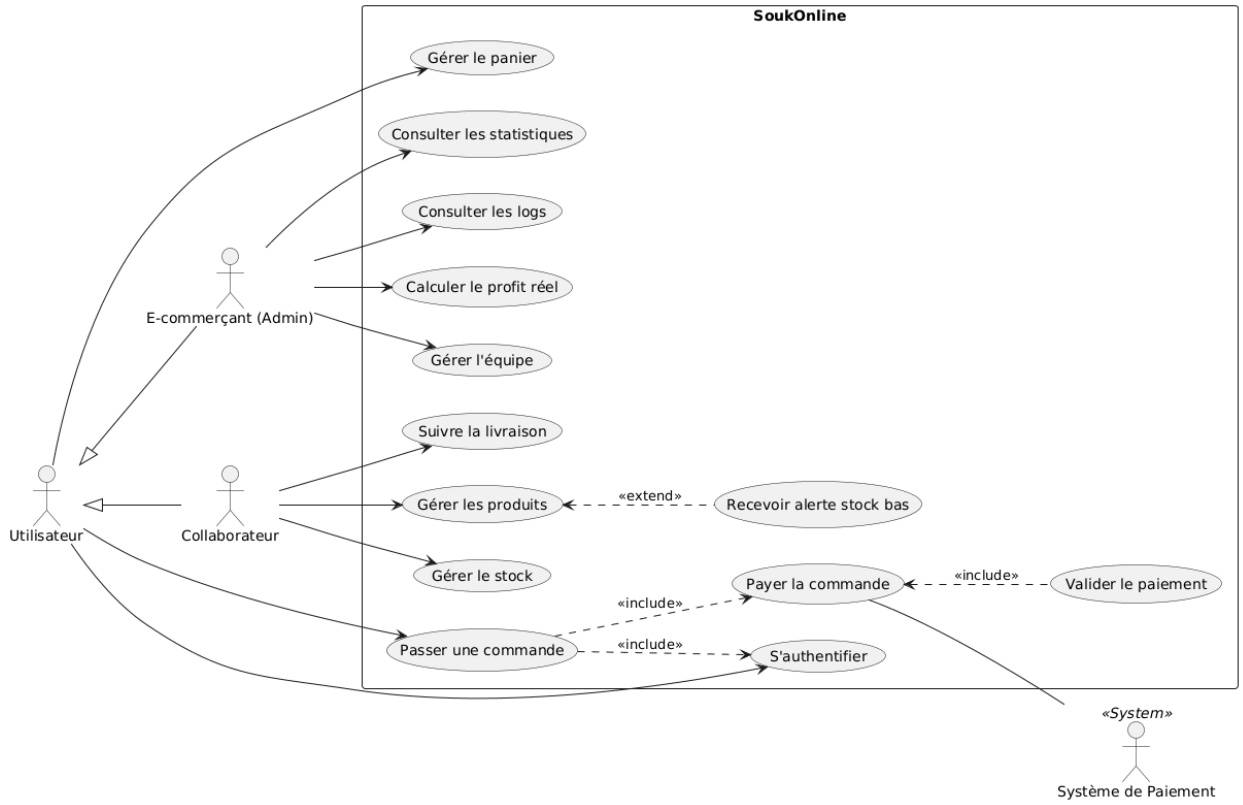
✓ Pull Request effectuée sur GitHub.

✓ Code Review validée par un collègue.



4. Modélisation UML :

Diagramme de cas d'utilisation (DCU) :



Description des cas d'utilisation :

Acteur	Cas d'utilisation	Description
Client	S'inscrire / Se connecter	Créer un compte ou accéder à l'espace client avec email et mot de passe.
Client	Gérer le panier	Ajouter des articles, modifier les quantités ou supprimer des produits.
Client	Passer command	Valider la sélection de produits et enregistrer les informations de livraison.

Système Paiement	Payer la commande	Effectuer la transaction financière via une simulation sécurisée.
Collaborateur	Suivre la livraison	Mettre à jour l'état d'avancement des colis (Expédié, Livré, Retour).
Collaborateur	Gérer le stock	Ajuster les quantités de produits disponibles et recevoir des alertes de stock bas.
Collaborateur	Gérer les produits	Ajouter de nouvelles fiches produits ou modifier les fiches existantes.
Administrateur	Calculer le profit réel	Analyser le bénéfice net après déduction des coûts d'achat, de livraison et de pub.
Administrateur	Gérer l'équipe	Créer des comptes pour les collaborateurs et définir leurs droits d'accès.
Administrateur	Voir statistiques	Consulter les graphiques de performance, les revenus et le taux de retour.
Administrateur	Consulter les logs	Vérifier l'historique détaillé des actions effectuées par chaque membre de l'équipe.

Diagramme de classes (DC) :

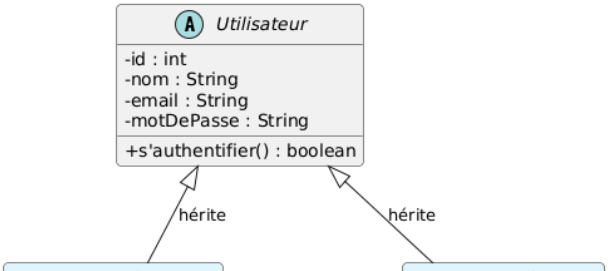
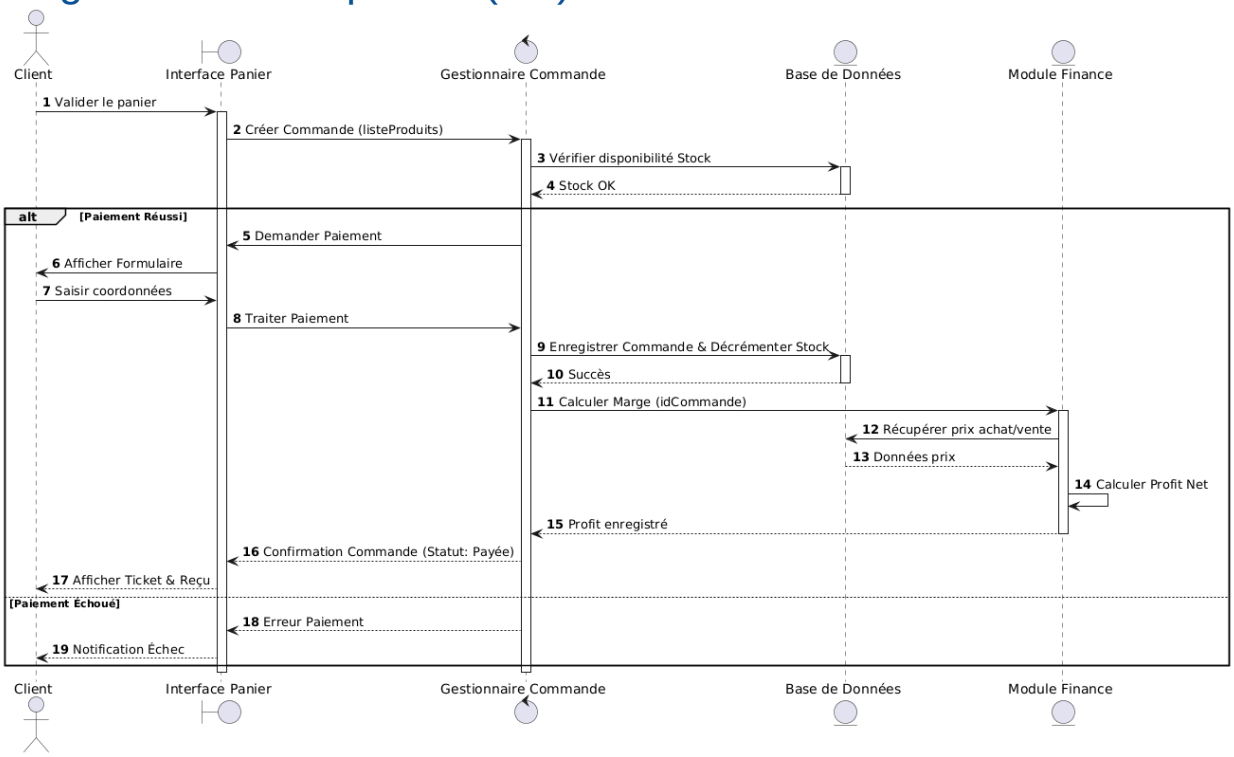


Diagramme de séquence (DS) — Processus de commande



Description du flux :

Phase d'Initialisation : Le **Client** valide son panier. L'**Interface Panier** envoie une requête de création de commande au **Gestionnaire de Commande** avec la liste des produits.

Vérification de Disponibilité : Avant toute transaction financière, le **Gestionnaire de Commande** interroge la **Base de Données** pour vérifier que les stocks sont suffisants.

Traitement du Paiement (Bloc 'alt' - Succès) :

- Le système affiche le formulaire de paiement.
- Le client saisit ses coordonnées, lesquelles sont traitées par le **Gestionnaire de Commande**.

Persistance et Mise à jour : Une fois le paiement validé, le système enregistre simultanément la commande et décrémente le stock en **Base de Données**.

Calcul Automatique de la Rentabilité : C'est ici que réside la valeur ajoutée de SoukOnline. Le **Gestionnaire de Commande** sollicite le **Module Finance**. Ce dernier récupère les prix d'achat et de vente en base, calcule le **Profit Net** en temps réel et l'enregistre.

Finalisation : Le client reçoit une confirmation de commande avec le statut "Payée" et son ticket/reçu s'affiche à l'écran.

Choix de conception :

Séparation des Responsabilités (SoC) :

L'utilisation d'un **Module Finance** distinct du **Gestionnaire de Commande** permet de découpler la logique de vente de la logique comptable. Cela facilite la maintenance : si tu souhaites modifier la formule de calcul du profit (ex: ajouter des taxes ou des frais de plateforme), tu ne modifies que le module Finance sans toucher au processus de vente.

Intégrité des Données et "Stock-First" :

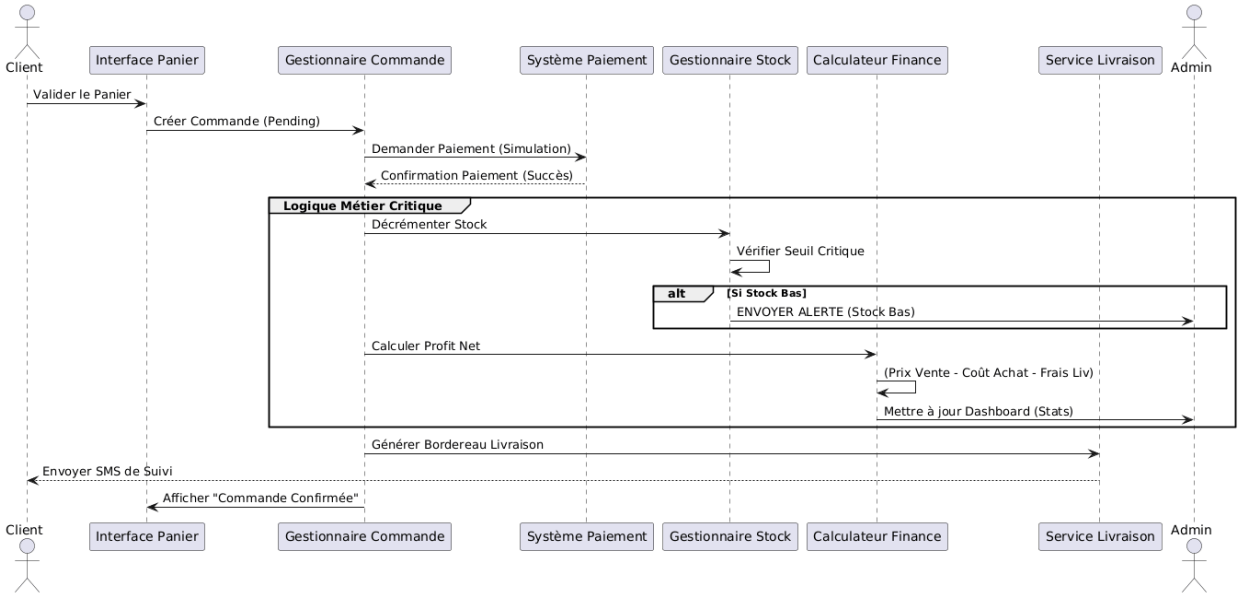
Le choix de vérifier le stock (étape 3) **avant** de demander le paiement est une règle métier critique. Cela évite les situations de "survente" (over-selling) où un client paierait pour un article qui vient de tomber en rupture de stock.

Automatisation du Profit & Loss (P&L) :

Contrairement aux systèmes e-commerce classiques où le marchand doit calculer ses marges sur Excel, la conception de SoukOnline intègre le calcul du profit (étapes 11 à 14) directement dans le flux transactionnel. Cela garantit que les données financières du dashboard sont **toujours à jour** et sans erreur humaine.

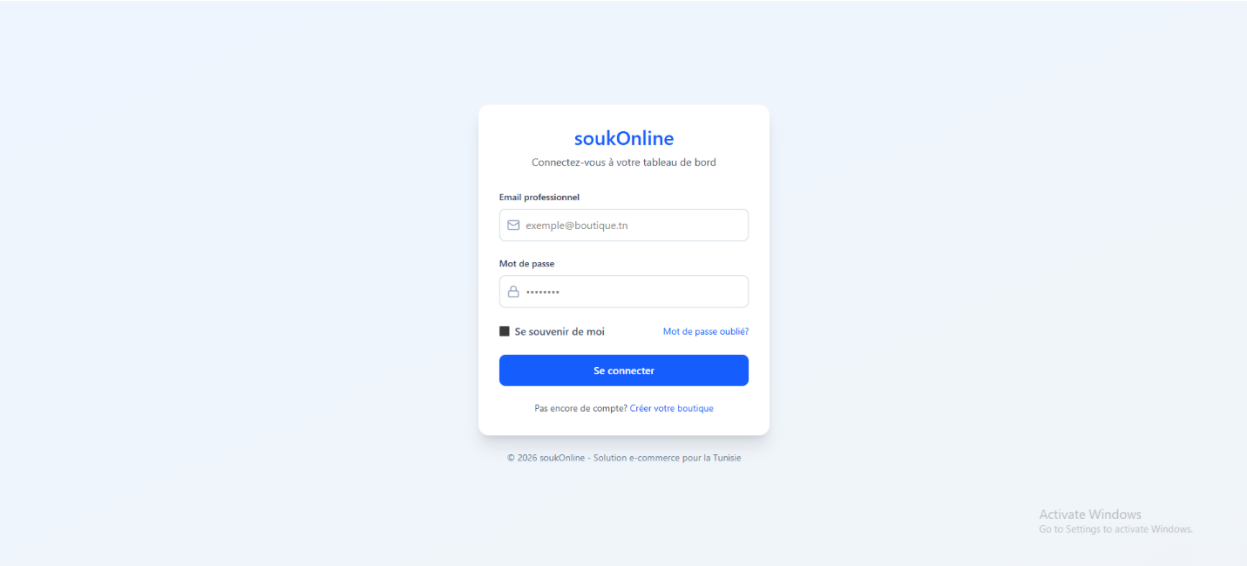
Gestion des Erreurs (Bloc Alternatif) :

L'utilisation d'un bloc alt dans le diagramme montre que le système est robuste : il prévoit explicitement le cas du **Paiement Échoué** (étapes 18-19). Dans ce cas, le stock n'est pas décrémenté et aucune donnée financière erronée n'est générée, assurant la cohérence du système.

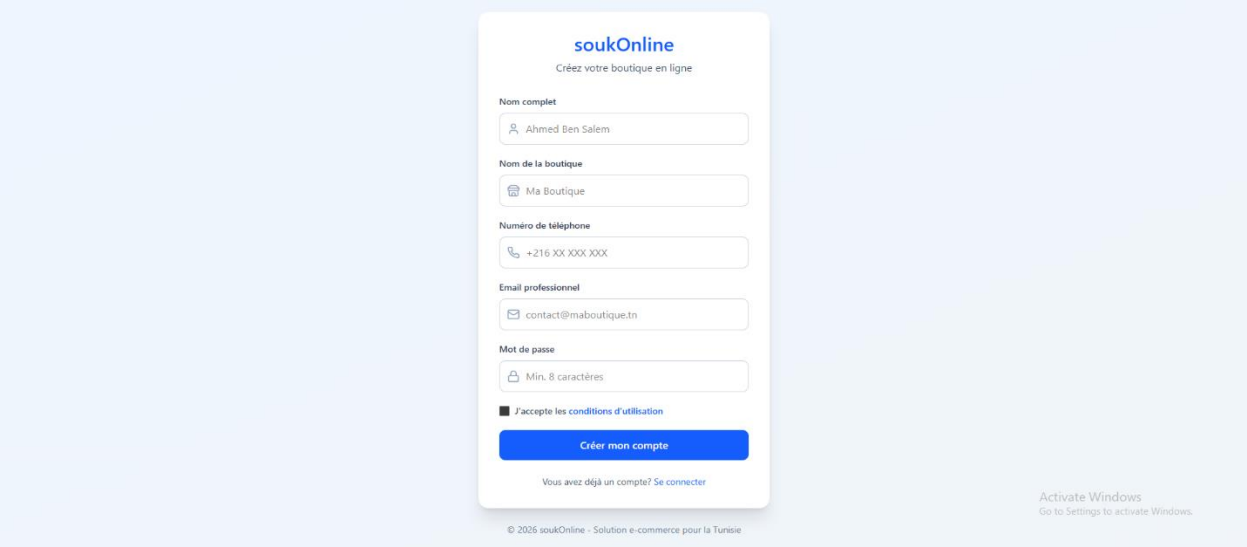


5. Maquettes Figma

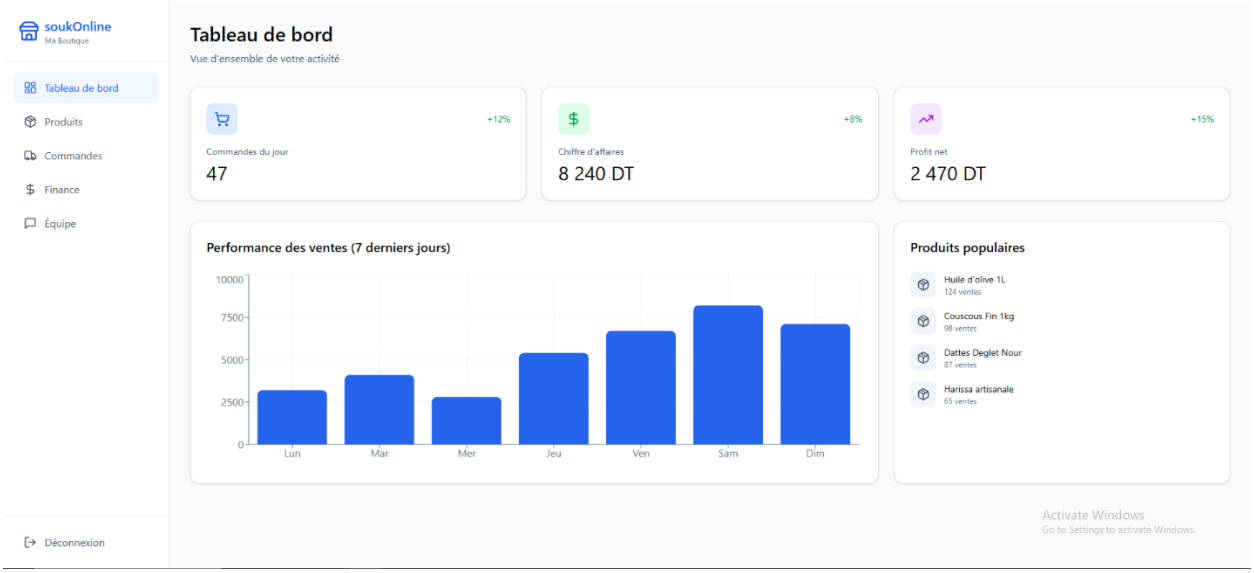
Captures d’écran:



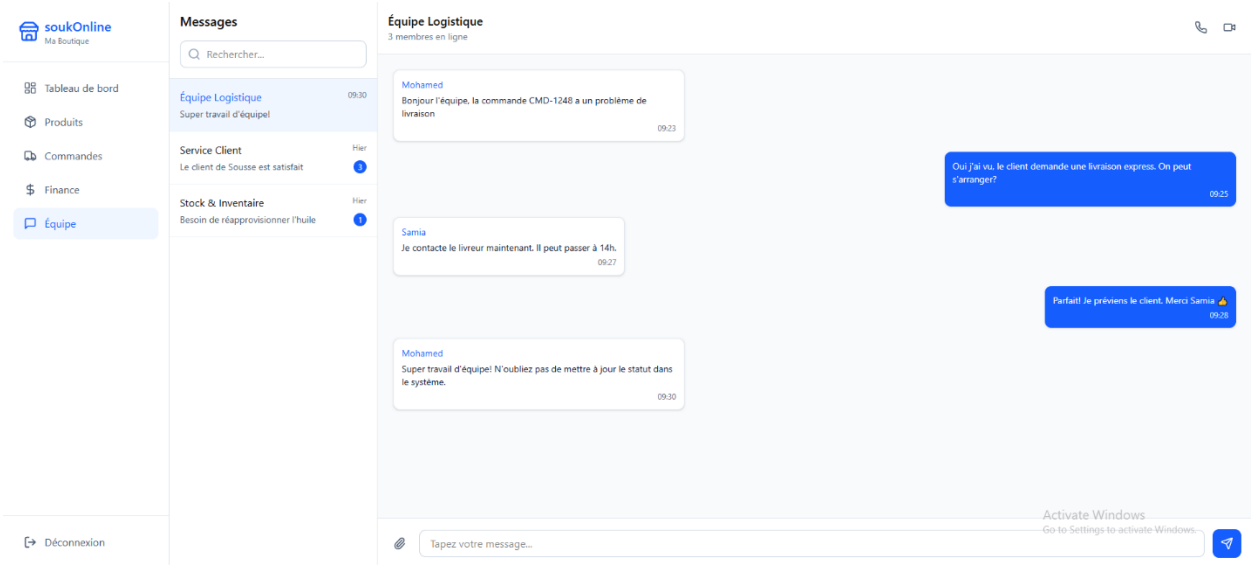
Sign-in Leila



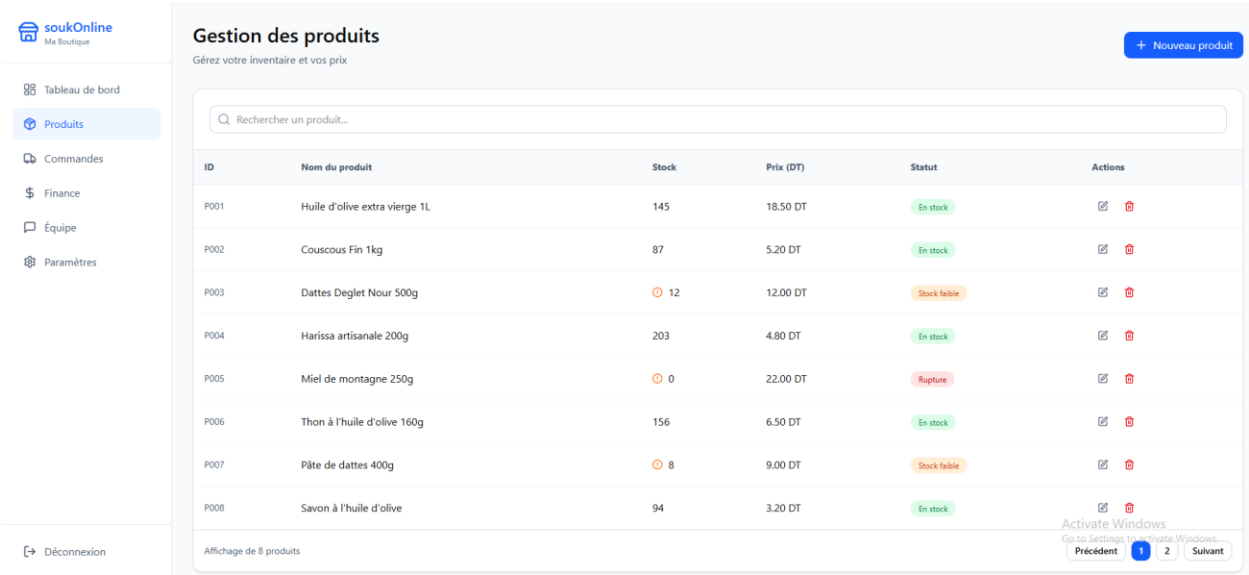
Log-in Leila



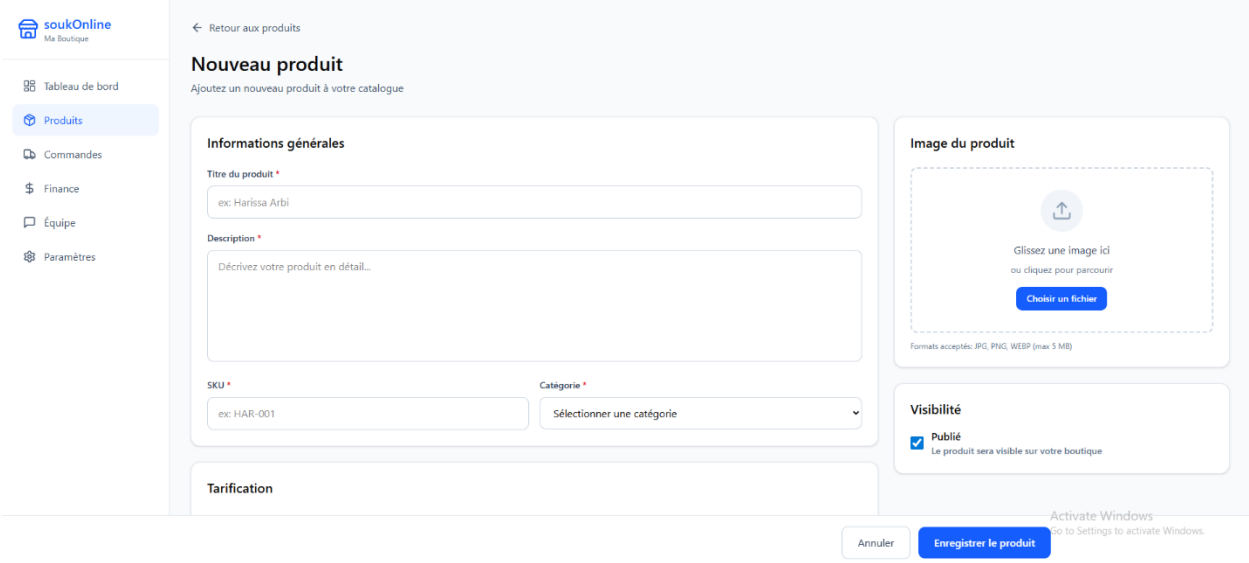
Vue d'ensemble Abdelkader



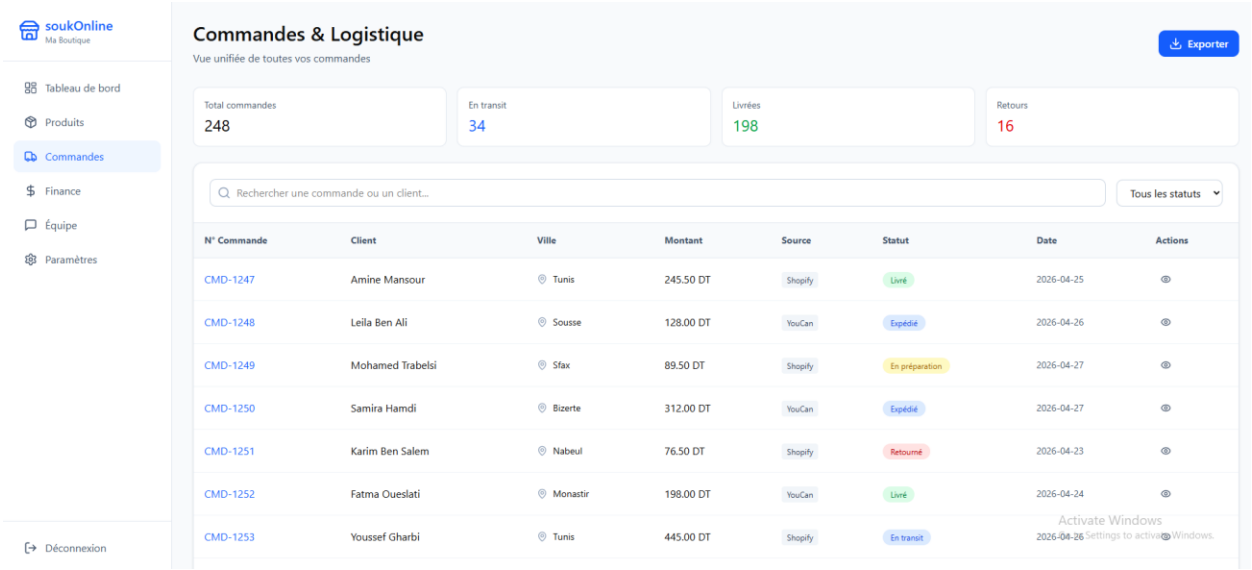
Hub de Communication Abdelkader



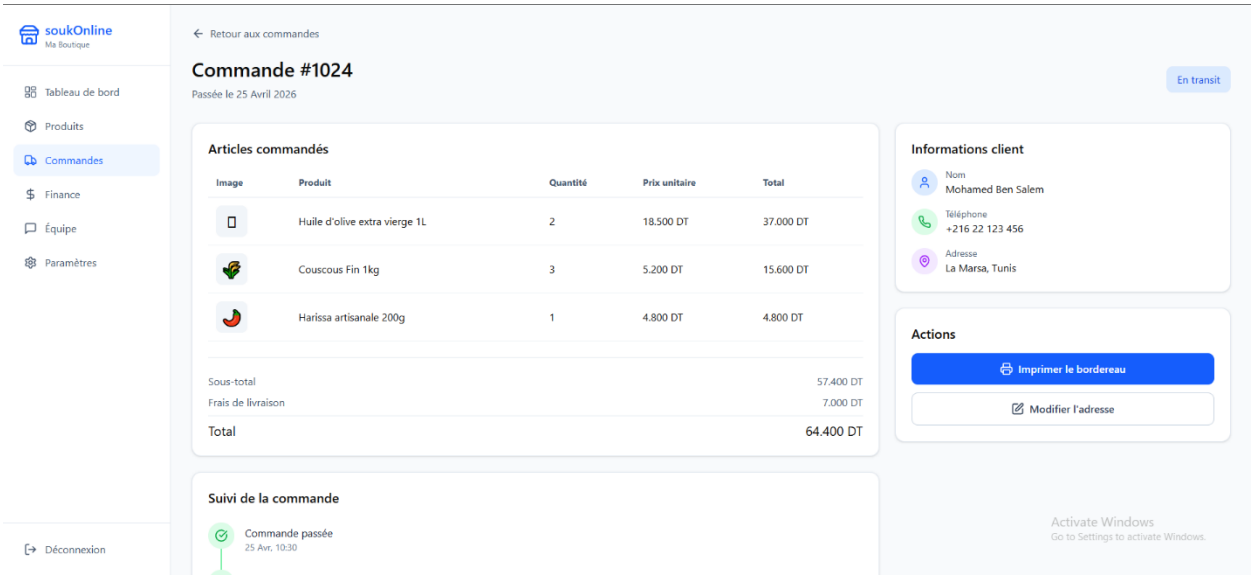
inventaire des produit Ghofrane



Formulaire d'Ajout/Édition Ghofrane



Liste des Commandes Centralisées Lina



Détail d'une Commande Lina

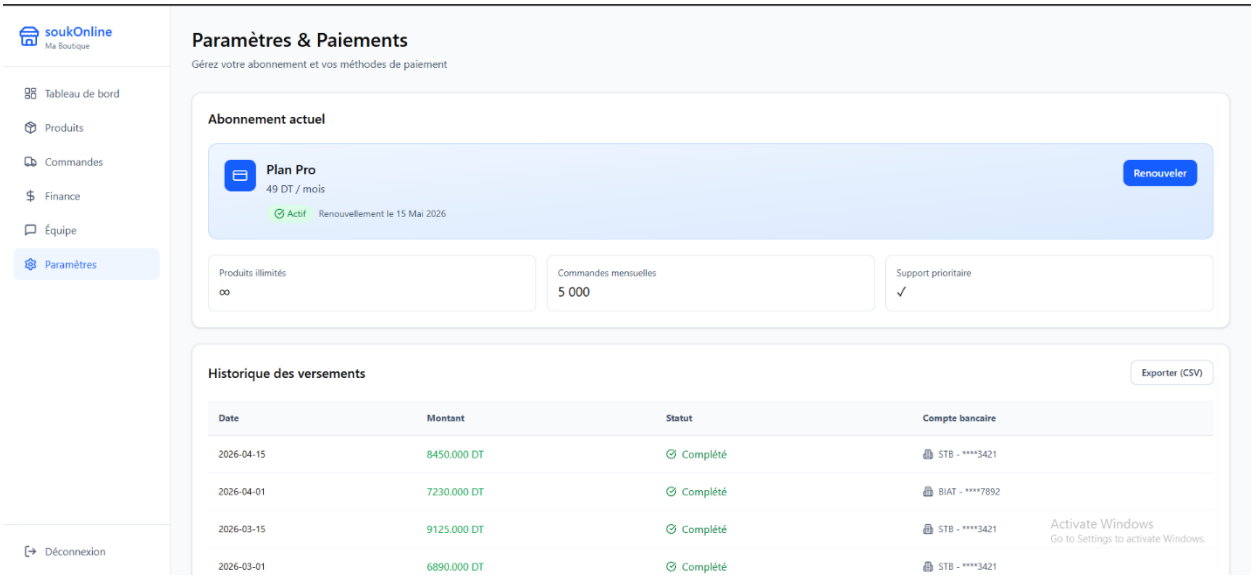
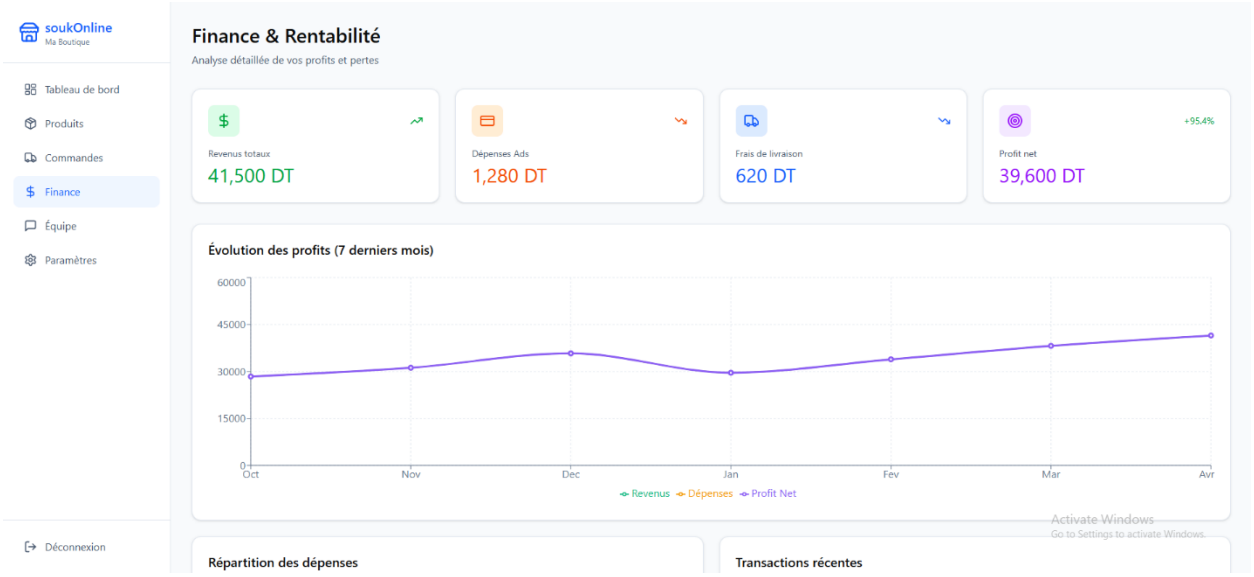


Tableau de Bord Financier (P&L) Hamza



Gestion des Paiements & Abonnements Hamza

Lien Figma (prototype cliquable) :

https://www.figma.com/make/3VR19IdR4vvHwBak3G3rWv/E-commerce-Management-Platform?t=9Mh75ejJTGpK8TTj-0&fullscreen=1&preview-route=%2Fproducts%2Fnew

Explication des parcours utilisateur

Parcours Client (Fluidité accrue) :

Point de différenciation : À l'étape **"Commande → Paiement"**, notre système déclenche instantanément le calcul automatique de la marge nette (via le module Finance), ce qui permet à l'admin de suivre son profit réel sans aucune action manuelle.

Parcours Administrateur (Centralisation) :

Point de différenciation : Le passage par le **"Dashboard"** ne sert pas uniquement à la gestion des produits. Il agrège en temps réel les données issues du flux client (stocks, commandes, revenus), transformant le simple outil de gestion en un véritable **cockpit de pilotage business**.

6. Code Approche Hybride

Méthodologie utilisée

Pour chaque fonctionnalité, nous avons suivi cette approche en 3 étapes :

Prompt IA : Rédiger un prompt précis pour générer le squelette.

Code généré : Récupérer et analyser le code produit par l'IA.

Code corrigé : Adapter, corriger et justifier les modifications.

Hamza v1 :

Élément	Contenu à insérer
Méthode ciblée	simulerPaiement() (Devenue traiterCommande pour l'interaction)
Prompt IA	"Améliore ma méthode simulerPaiement pour qu'elle vérifie si le produit est en stock avant de valider, et qu'elle diminue le stock si le paiement réussit."
Code généré par l'IA	https://drive.google.com/file/d/104pbc9m46-l0aPJk0HFNNRKeu42C1oE7/view?usp=drive_link
Code final corrigé	https://drive.google.com/file/d/1p_PJl4xRp4S_0inLT0qOOJ6t0XdcE2Bb/view?usp=drive_link
Justification	J'ai ajouté un paramètre Produit à la méthode pour créer une interaction réelle entre mon module et celui de Ghofrane. J'ai aussi ajouté des conditions de sécurité.

Hamza v2 :

Élément	Contenu à insérer

Méthode ciblée	calculerMargeNette()
Prompt IA	"Génère une classe Finance en Java qui calcule le profit net en soustrayant les dépenses publicitaires et les coûts de livraison du revenu total."
Code généré par l'IA	https://drive.google.com/file/d/19p56ebPW0UhMRcVAKTcuaSkiWmx3BUZW/view?usp=drive_link
Code final corrigé	https://drive.google.com/file/d/1MUTw3i3lcVr1evQ1k_W2Nz_SCbbwMknH/view?usp=drive_link
Justification	J'ai ajouté une variable soldeBoutique qui se met à jour à chaque calcul pour que le Dashboard puisse afficher l'historique financier.

Ghofrane v1 :

Élément	**Contenu à insérer**
Méthode ciblée	**verifierStock() (Devenue reduireStock pour l'interaction)**
Prompt IA	**"Génère une classe Produit en Java avec gestion de stock, seuil d'alerte et une méthode de recherche par nom pour un catalogue e-commerce."**
Code généré par l'IA	**https://drive.google.com/file/d/1VNYthVjEACDEo4pzmRn1NC9jYzesmPf1/view?usp=drive_link**
Code final corrigé	**https://drive.google.com/file/d/1vpO5ef_izlpBzUrhF_gRY4fwjtGdSSyW/view?usp=drive_link**
Justification	**J'ai transformé la méthode verifierStock pour qu'elle puisse soustraire réellement la quantité vendue. J'ai aussi ajouté des setters spécifiques pour permettre à Hamza (Paiement) de modifier le stock à distance.**

Ghofrane v2 :

Élément	**Contenu à insérer**
Méthode ciblée	**ajouterProduitALaCategorie()**
Prompt IA	**"Ajoute une liste de produits à ma classe Categorie en Java et crée une méthode pour ajouter un produit à cette liste."**

Code généré par l'IA	https://drive.google.com/file/d/1724ueJI26nOI8zyRYXxe1dk68-7i2FkV/view?usp=drive_link
Code final corrigé	https://drive.google.com/file/d/1ySdsQuwLqnX2XyZ2sA93afAve9fbMjv-/view?usp=drive_link
Justification	**J'ai ajouté une ArrayList pour stocker les objets Produit. Cela permet de lier réellement les deux classes de Ghofrane entre elles (Interaction interne).**

Leila v1 :

Élément	**Contenu à insérer**
Méthode ciblée	**seConnecter()**
Prompt IA	**"Génère une classe Utilisateur en Java avec id, email, mot de passe et une méthode de connexion de base qui vérifie si les champs sont vides."**
Code généré par l'IA	https://drive.google.com/file/d/1ABF-8l1l4397UQs4Qly66UkrXON1Oj9H/view?usp=drive_link
Code final corrigé	https://drive.google.com/file/d/1MCuzO6oHGYVZMu95pU_gwrpEaOhDSsbP/view?usp=drive_link
Justification	**J'ai renforcé la sécurité : la méthode vérifie maintenant si l'email contient un "@" et si le mot de passe correspond exactement à celui stocké. J'ai aussi ajouté un message de bienvenue personnalisé.**

Leila v2 :

Élément	**Contenu à insérer**
Méthode ciblée	**ajouterAchat()**
Prompt IA	**"Crée une classe Client en Java qui hérite d'Utilisateur, avec une adresse de livraison et une méthode pour afficher ses informations."**
Code généré par l'IA	https://drive.google.com/file/d/1XDIInpZw5IbfTozbzhcMUyYGwEWLohMm/view?usp=drive_link

Code final corrigé	**https://drive.google.com/file/d/1Nt40-bnUVYDHX5RBb-44rV4Q4yM9DiQW/view?usp=drive_link**
Justification	**J'ai ajouté un système de points de fidélité. À chaque achat validé, le client gagne des points. C'est une interaction directe avec le module de Paiement.**

Abdelkader v1et v2 :

Élément	**Contenu à insérer**
Méthodes ciblées	**genererRapportJournalier() et afficherResume()**
Prompt IA	**"Génère une classe Dashboard et une classe Statistique en Java pour un administrateur e-commerce, incluant des méthodes pour afficher l'évolution des ventes et générer des rapports."**
Code généré par l'IA	**https://drive.google.com/drive/folders/1elHkyZm0XW3P1idWN4gErzQrFlOUo2pO?usp=drive_link**
Code final corrigé	**https://drive.google.com/drive/folders/177bfGd8L_QiBtHqosJaJL24kNscUcYJI?usp=drive_link**
Justification	**J'ai lié le Dashboard au module Finance de Hamza. Au lieu d'afficher des chiffres fixes ("1700 DT"), le rapport calcule maintenant les données en temps réel en interrogeant la classe Finance.**

Lina V1 V2 :

Élément	**Contenu à insérer**
Méthodes ciblées	**ajouterAuPanier() et mettreAJourStatutLivraison()**
Prompt IA	**"Génère deux classes Java : Panier (avec liste d'articles et calcul de frais de port par ville) et Livraison (avec suivi et mise à jour de statut) pour un site d'e-commerce."**
Code généré par l'IA	**https://drive.google.com/drive/folders/1Vzb9wSN3uNFQrZipvr0janrMmNxTLE2e?usp=drive_link**

Code final corrigé	https://drive.google.com/drive/folders/1Bh65v7oIO0d1Di1kQcW-QZvKdR2Lmeut?usp=drive_link
Justification	**J'ai transformé la liste de chaînes de caractères (String) du panier en une liste d'objets Produit pour permettre une interaction réelle avec le catalogue de Ghofrane. J'ai aussi lié la livraison au paiement de Hamza.**

Conclusion : L'approche hybride AGL + IA est la plus efficace : l'IA génère rapidement le squelette, l'équipe corrige, adapte et comprend le code. Cette méthode nous a permis de gagner du temps tout en maintenant la qualité et la compréhension du code.

7. Tableau comparatif AGL vs IA

Critère	Atelier de Génie Logiciel (Sprint 1)	Intelligence Artificielle (Sprint 2)
Vitesse de démarrage	Lente (configuration et modélisation requises).	Instantanée (génération de code immédiate).
Structure du code	Très rigoureuse (respect des standards POO).	Variable (nécessite souvent un recadrage).
Logique Métier	Limitée aux signatures de méthodes.	Capable de générer des algorithmes complexes.
Interactivité	Faible (génère des classes isolées).	Elevée (peut lier des concepts si bien promptée).
Fiabilité	Totale (code squelette sans erreur).	Modérée (erreurs logiques ou syntaxiques possibles).
Maintenance	Facile grâce à la structure claire.	Difficile sans une relecture humaine approfondie.

8. Sprint Reviews & Rétrospectives

Sprint 1

- **Review :** Présentation du squelette Java, des maquettes Figma et de la base de données. Validation des user stories liées à l'authentification.
- **Rétrospective :** * *Points positifs :* Bonne répartition des tâches et compréhension rapide des diagrammes UML.
    - *Axes d'amélioration :* Besoin de mieux définir les interfaces pour faciliter l'intégration future.

Sprint 2

- **Review :** Démonstration du flux complet (commande -> livraison -> calcul de profit). Validation de l'intégration de la logique métier.
- **Rétrospective :**
    - *Points positifs :* Communication fluide entre les membres de l'équipe.
    - *Vitesse :* La vélocité réalisée a dépassé la vélocité prévue grâce à une meilleure maîtrise des concepts POO en Java.

Tu peux placer ce bloc juste après la section **"Méthode Agile"** pour conclure ton document.

Petit conseil de peer : Pour rendre ton GitHub encore plus "pro", tu pourrais ajouter une petite phrase sur la **vélocité** (mentionnée dans ton image) sous forme de stats simples :

Vélocité du projet :

- Prévue : 25 Story Points
- Réalisée : 28 Story Points (Efficacité accrue au Sprint 2)

9. Bilan

Leila (Product Owner + Dev)

"En tant que PO, j'ai appris à prioriser les besoins des e-commerçants tunisiens pour simplifier leur quotidien. Ma contribution principale a été la conception de l'authentification sécurisée. La plus grande difficulté fut de faire le pont entre la vision métier et les contraintes techniques du Java, mais cette expérience m'a permis de mieux comprendre le cycle de vie complet d'un produit."

Hamza (Scrum Master + Dev)

"Mon rôle était de garantir la fluidité du développement. J'ai géré le système de paiement et la logique de calcul des marges. J'ai appris que la communication est le pilier d'une équipe Agile. La difficulté était de gérer les imprévus techniques lors des sprints, mais cela a renforcé ma capacité à résoudre des problèmes complexes sous pression."

Ghofrane (Développeuse)

"J'ai développé le catalogue produit et les systèmes de filtres. Apprendre à structurer les données en POO m'a permis d'améliorer ma rigueur de code. Ma difficulté majeure était de rendre la recherche fluide, mais en collaborant avec l'équipe, nous avons trouvé une solution efficace qui rend le catalogue très intuitif."

Lina (Développeuse)

"Je me suis concentrée sur la gestion du panier et le suivi des commandes. Ce projet m'a permis de concrétiser mes connaissances en Java. Le défi était de synchroniser l'état du panier avec la commande finale. J'ai appris à mieux utiliser GitHub pour éviter les conflits lors des intégrations, ce qui est une compétence précieuse."

Abdelkader (Développeur)

"J'ai pris en charge le Dashboard administrateur et les statistiques de vente. La complexité résidait dans l'agrégation des données pour offrir une vision claire des profits. J'ai énormément appris sur l'architecture logicielle. Ma difficulté était la visualisation des données en console, mais cela m'a poussé à structurer mon code pour qu'il soit plus maintenable et lisible."